

WolfCafe - Use Case 8

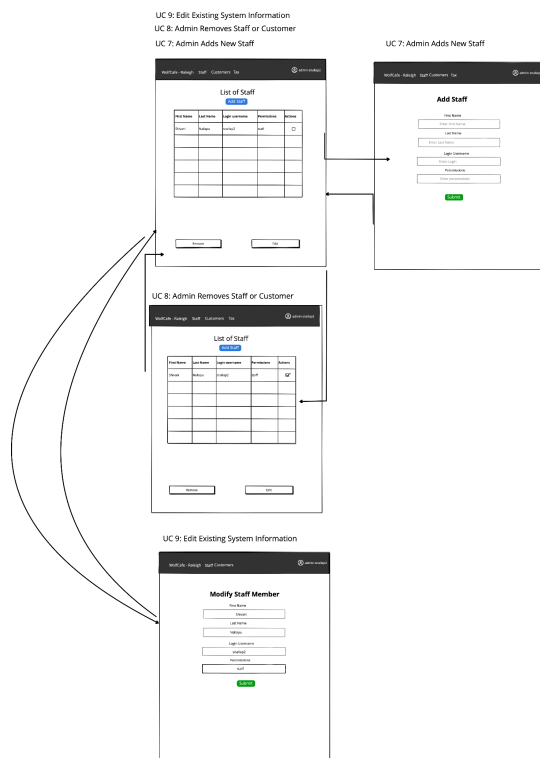
“Admin removes existing user from the system”

Design Decisions (frontend, backend) - done

Administration tasks such as removing users requires us to implement substantial elements in our design. For our UML diagram, we considered new repositories to contain the role of a given user (**'role repository'**) [**'regular'**, **'staff'**, **'administrator'**] as well as needing a repository to store the users themselves (**'user repository'**). We manage all of this in the backend through an authentication controller we designed (denoted **'AuthController'** class), which allows for the registration, handling of authentication and deleting of users. Adjacently, we also consider we need a service to handle more complex operations not suited for being included directly in the controller class. We denote this as our **'AuthServiceImpl'** class, which contains several helper methods for the aforementioned class. We also had to consider how this would affect our frontend React components. We concluded on handling user/role data by utilizing React states, and having a singular frontend component (**'StaffList.jsx'**) for management of staff members in a given location. We concluded that for ease of use, administrators may need to sort through a long list of staff depending on the size of a given location. For this reason, we implemented searching functionalities as well as a modular list tabular component for ease of view. We show staff information in each table row, as well as action buttons (**'delete'**, **'edit'**...). This does affect our database, but given we are using **Hibernate** (our ORM tool) to map objects to our relational database (**MySQL**), our DTO (**'UserDto'**, **'RoleDto'**) and entity objects (**'User'**, **'Role'**) allow us to easily manage these entries in our service implementations. In Hibernate, we will need **many-to-one** relationships as users will only have one role, and roles contain infinitely many users. To handle large lists of staff, we introduce our decision of **server-side pagination** to improve experience for administration, improving server responsiveness. Testing will be done using the **Selenium** framework on the front-end, with **JUnit** being used in the backend to test our repositories, controllers and services. We also consider multiple edge-cases we need to consider in our back-end, such as preventing the deletion of the final administrator user to prevent an integrity violation in the database.

Frontend Elements

Original Design (**SAME AS UPDATED DESIGN**)

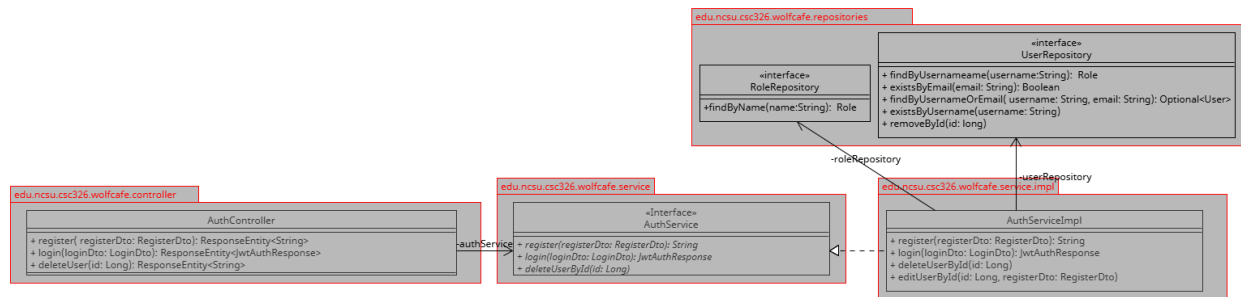


Changelog

- Since it's iteration 1A we did not change anything

Backend Elements

Class Diagrams



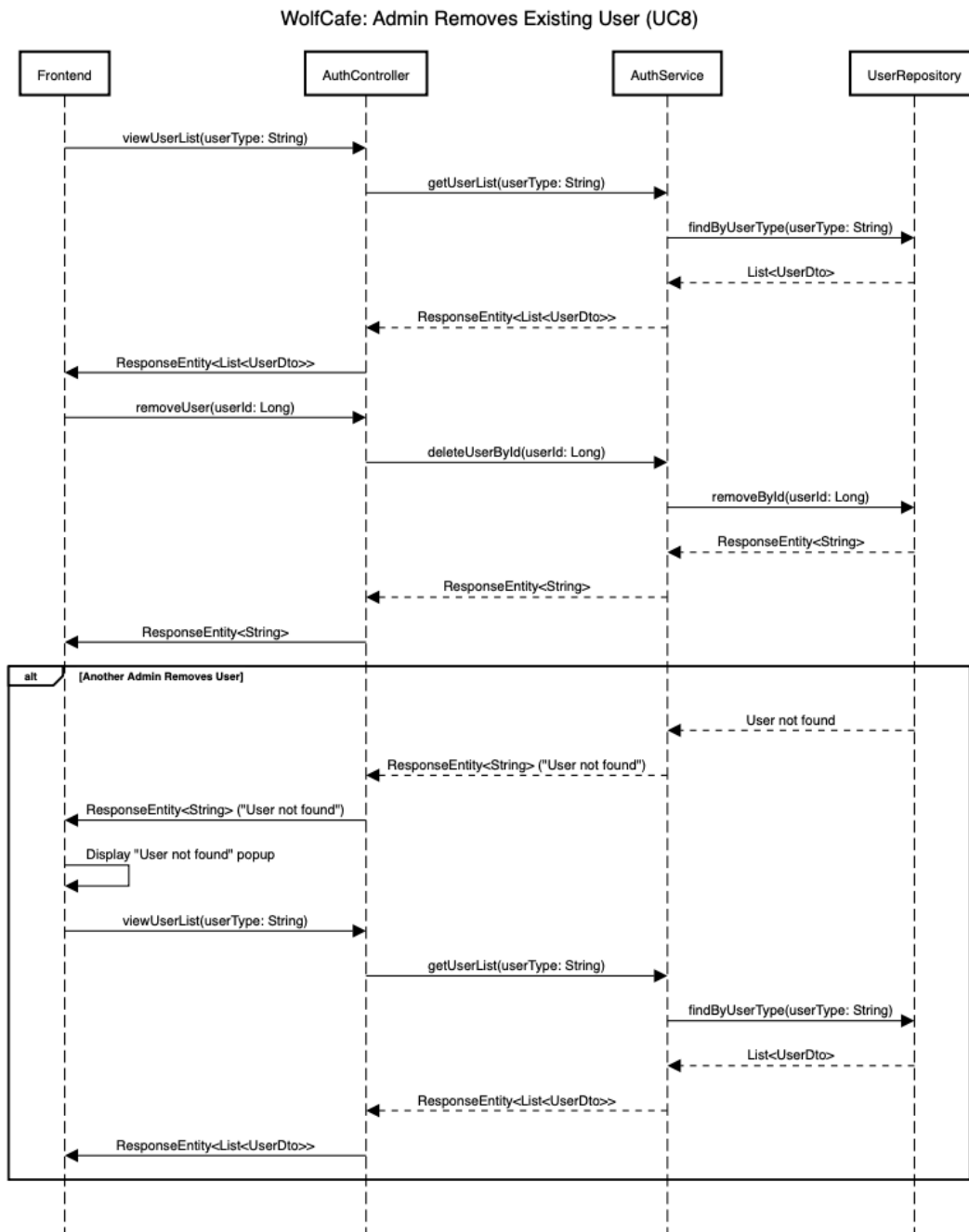
Original Design (**SAME AS UPDATED DESIGN**)

Changelog

- Since it's iteration 1A we did not change anything

Sequence Diagram:

Original Design (**SAME AS UPDATED DESIGN**)



Changelog

- Since it's iteration 1A we did not change anything